Relational Database Management System

Module 1

Jan 2026-May 2026

Introduction

- Introduction
- Characteristics of databases
- Comparison of File system and Database approach
- Users of Database system
- Concerns when using an enterprise database
- Data Independence
- DBMS system architecture
- Database Administrator

Drawbacks of using file systems to store data

- Data redundancy and inconsistency
  - Multiple file formats, duplication of information in different files
- Difficulty in accessing data
  - Need to write a new program to carry out each new task
- Data isolation
  - Multiple files and formats
- Integrity problems
  - It ensures that data remains correct and valid throughout its lifecycle, from creation to modification and deletion.
  - Hard to add new constraints or change existing ones

Drawbacks of using file systems to store data (Cont.)

- Concurrent access by multiple users
  - Concurrent access needed for performance
  - Uncontrolled concurrent accesses can lead to inconsistencies
    - Example: Two people reading a balance (say 100) and updating it by withdrawing money (say 50 each) at the same time
- Security problems
  - Hard to provide user access to some, but not all, data

Database systems offer solutions to all the above problems

DBMS

- **Database**: A database is an organized collection of data.
- **DBMS:** A database management system (DBMS) is system software for creating and managing databases.
  - DBMS is Collection of interrelated data
  - It has set of programs to access the data
  - DBMS provides an environment that is both convenient and efficient to use
  - DBMS contains information about a particular enterprise
  - **Examples**: Microsoft Access, File-based systems like XML, CSV.

- **RDBMS (Relational Database Management System)**

- **Definition**: An RDBMS is a type of DBMS that stores data in a structured format using **tables** and enforces **relationships** between these tables through keys (e.g., primary keys and foreign keys).
- **Data Storage**: Data is stored in tables, and relationships between tables are strictly defined.
- **Examples**: MySQL, PostgreSQL, Oracle Database, Microsoft SQL Server.

Advantages of DBMS

1. Redundancy reduced: As there is centralized database redundancy is reduced.
2. Less Inconsistency: As redundancy is reduced data inconsistency will be reduced
3. More Integrity: Integrity constraints can be added so data will be more accurate
4. No Data Isolation: As there is centralized database
5. More Security: Privileges can be given to the users and data can be secured
6. Concurrent access by multiple users: Controlled concurrent access to multiple users is possible to increase the performance.

Example

- Table: Student

Name	Roll Number	Class	Div
John Smith	101	SY	C
Emma Johnson	102	SY	C
Liam Williams	103	SY	C

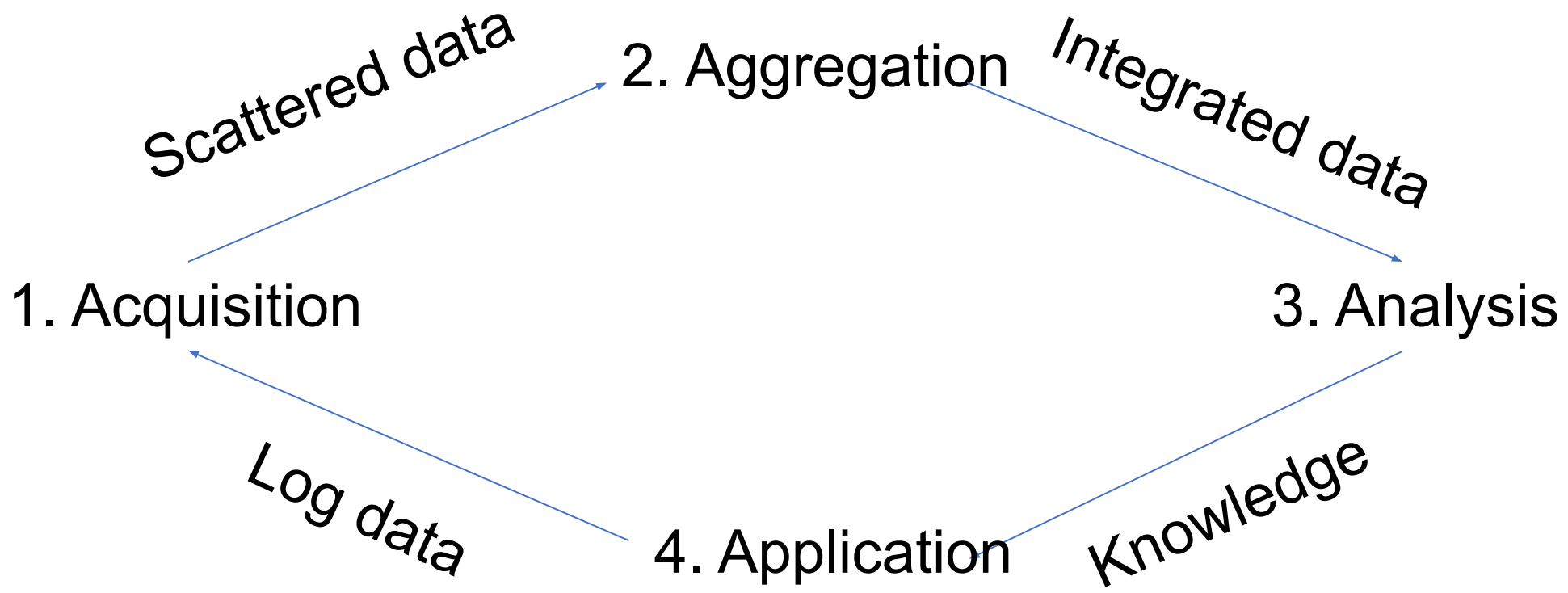
- Table: Student_marks

Roll Number	Subject	Marks
101	DBMS	85
101	OS	90
102	DBMS	78
102	OS	88
103	DBMS	78
103	OS	85

Importance of “big data”

- Government
- Private Sector
  - Walmart handles more than 1 million customer transactions every hour, which is imported into databases estimated to contain more than 2.5 petabytes of data
  - Facebook handles 40 billion photos from its user base
  - Falcon Credit Card Fraud Detection System protects 2.1 billion active accounts world-wide
- Science
  - Large Synoptic Survey Telescope will generate 140 Terabyte of data every 5 days
  - Biomedical computation like decoding human Genome and personalized medicine

Lifecycle of Data: 4 “A”s



Basic Definitions

- **Database:**
  - A collection of related data.
- **Data:**
  - Known facts that can be recorded and have an implicit meaning.
- **Database Management System (DBMS):**
  - A software package/system to facilitate the creation and maintenance of a computerized database.
- **Database system:**
  - The DBMS software together with the data itself. Sometimes, the applications are also included.

Applications interact with a database by generating

- **Queries:** that access different parts of data and formulate the result of a request

- **Transactions**: that may read some data and “update” certain values or generate new data and store that in the database

Characteristics of the Database Approach

- **Self-describing nature of a database system:**
  - A DBMS **catalog** stores the description of a particular database (e.g. data structures, types, and constraints)
  - The description is called **meta-data***.
  - This allows the DBMS software to work with different database applications.
- **Insulation between programs and data:**
  - Called **program-data independence**.
  - Allows changing data structures and storage organization without having to change the DBMS access programs

Characteristics of the Database Approach (..contd)

- **Data abstraction:**
  - A **data model** is used to hide storage details and present the users with a conceptual view of the database.
  - Programs refer to the data model constructs rather than data storage details
- **Support of multiple views of the data:**
  - Each user may see a different view of the database, which describes **only** the data of interest to that user.

Characteristics of the Database Approach (..contd)

- **Sharing of data and multi-user transaction processing:**
  - Allowing a set of **concurrent users** to retrieve from and to update the database.
  - *Concurrency control* within the DBMS guarantees that each transaction is correctly executed or aborted
  - *Recovery* subsystem ensures each completed transaction has its effect permanently recorded in the database
  - **OLTP** (Online Transaction Processing) is a major part of database applications; allows hundreds of concurrent transactions to execute per second.

Advantages of Using the Database Approach

- Controlling redundancy in data storage and in development and maintenance efforts.
  - Sharing of data among multiple users.
- Restricting unauthorized access to data. Only the DBA staff uses privileged commands and facilities.
- Providing persistent storage for program Objects
  - E.g., Object-oriented DBMSs make program objects persistent
  - Providing storage structures (e.g. indexes) for efficient query
- Providing optimization of queries for efficient processing
- Providing backup and recovery services
- Providing multiple interfaces to different classes of users
- Representing complex relationships among data
- Enforcing integrity constraints on the database
- Drawing inferences and actions from the stored data using deductive and active rules and triggers

Advantages of Using the Database Approach

- Potential for enforcing standards:
  - **Standards** refer to data item names, display formats, screens, report structures, meta-data (description of data), Web page layouts, etc.
- Reduced application development time:
  - Incremental time to add each new application is reduced.
- Flexibility to change data structures:
  - Database structure may evolve as new requirements are defined.
- Availability of current information:
  - Extremely important for on-line transaction systems such as shopping, airline, hotel, car reservations.
- Economies of scale:
  - Wasteful overlap of resources and personnel can be avoided by consolidating data and applications across departments.

Limitations of the Database Approach

- Main inhibitors (costs) of using a DBMS:
  - High initial investment and possible need for additional hardware
  - Overhead for providing generality, security, concurrency control, recovery, and integrity functions
- When a DBMS may be unnecessary:
  - If the database and applications are simple, well defined, and not expected to change
  - If access to data by multiple users is not required
- When a DBMS may be infeasible
  - In embedded systems where a general-purpose DBMS may not fit in available storage
- When no DBMS may suffice:
  - If the database system is not able to handle the complexity of data because of modeling limitations
  - If the database users need special operations not supported by the DBMS

Data Schemas and Database Instance

- **Database Schema:**
  - The ***description*** of a database.
  - Includes descriptions of the database structure, data types, and the constraints on the database.

  Example:

  BookID INT PRIMARY KEY, Title VARCHAR(255) NOT NULL, Author VARCHAR(255), Publisher VARCHAR(255), YearPublished INT

Schema Diagram:

An ***illustrative*** display of (most aspects of) a database schema.

Schema Construct:

- A schema construct refers to the various elements or components that define the structure of a database schema.
- These constructs collectively represent how the data is organized, stored, and related within the database.
- In simpler terms, schema constructs are the building blocks of a database schema, such as tables, fields, keys, relationships, and constraints.

Data Schemas and Database Instance

Database State / database instance (or occurrence or snapshot):

The actual data stored in a database at a ***particular moment in time***. This includes the collection of all the data in the database.

The term *instance* is also applied to individual database components, e.g. *record instance, table instance, entity instance*

Data Schemas and Database Instance

Database State / database instance (or occurrence or snapshot):

- Database State:
  - Refers to the content of a database at a moment in time.
- Initial Database State:
  - Refers to the database state when it is initially loaded into the system.
- Valid State:
  - A state that satisfies the structure and constraints of the database.
- Distinction
  - The ***database schema*** changes very infrequently.
  - The ***database state*** changes every time the database is updated.
- **Schema** is also called **intension**.
- **State** is also called **extension**.

Data Schemas

STUDENT

Name	Student_number	Class	Major

COURSE

Course_name	Course_number	Credit_hours	Department

PREREQUISITE

Course_number	Prerequisite_number

SECTION

Section_identifier	Course_number	Semester	Year	Instructor

GRADE_REPORT

Student_number	Section_identifier	Grade

Database Instance

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	04	King
92	CS1310	Fall	04	Anderson
102	CS3320	Spring	05	Knuth
112	MATH2410	Fall	05	Chang
119	CS1310	Fall	05	Anderson
135	CS3380	Fall	05	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

Database Users

Actors on the Scene

- ★ Database Administrators
- ★ Database Designers
- ★ End Users
- ★ System Analysts & Application Programmers (Software Engineers)

Workers Behind the Scene

- ★ System designers & implementers
- ★ Tool developers
- ★ Operators & maintenance personnel

Database users

- **Users may be divided into**
  1. Those who actually use and control the database content, and those who design, develop and maintain database applications (called "*Actors on the Scene*")

  2. Those who design and develop the DBMS software and related tools, and the computer systems operators (called "*Workers Behind the Scene*").

Database users: Actors on the scene

Database Administrator

- Coordinates all activities related to database systems
- Has good understanding of the enterprise’s information
- Administrator's duties include:
  - Schema definition
  - Storage structure and access method definition
  - Schema and physical organization modification
  - Granting user authorities to access database
  - Specifying integrity constraints
  - Acting as linked with users
  - Monitoring performance and responding to changes in requirements
- **Example:**
- A DBA at an e-commerce company creates user accounts for employees, grants permissions, and schedules daily backups to prevent data loss.

Database users: Actors on the scene

Database designers

- Identifying the data to be stored in the database.
- Communicating with database users , understand their requirements and design database.
- Database designers create the structure of the database, such as the schema, tables, relationships, and constraints.
- They ensure that the database is optimized for performance and can handle the required data efficiently.
- **Example:**
- A database designer for a hospital management system defines tables for Patients, Doctors, and Appointments, ensuring the appropriate relationships between them.

Database users: Actors on the scene

- **End-users:** They use the data for queries, reports and some of them update the database content.
- **End-users can be categorized into**:

  1. Casual: access database occasionally when needed.

  Example: A manager queries the sales database to generate a monthly revenue report.

  2. Naïve or parametric:

  - They use previously well-defined functions in the form of "canned transactions" against the database.
    - Regularly perform predefined operations using specific applications.
    - Users of mobile apps mostly fall in this category
    - A bank teller uses software to fetch and update customer account details.
    - Social media users post and read information from websites

Database users: Actors on the scene

3. Sophisticated:

- Many use tools in the form of software packages that work closely with the stored database.
- Use advanced tools or write queries directly.
  Example: A data analyst uses SQL to extract trends from customer purchase data.

4. Stand-alone:

- Mostly maintain personal databases using ready-to-use packaged applications.
- *Example:* A researcher uses MS Access for managing survey data.
- Another example is a user that maintains a database of personal photos and videos.

Database users: Actors on the scene

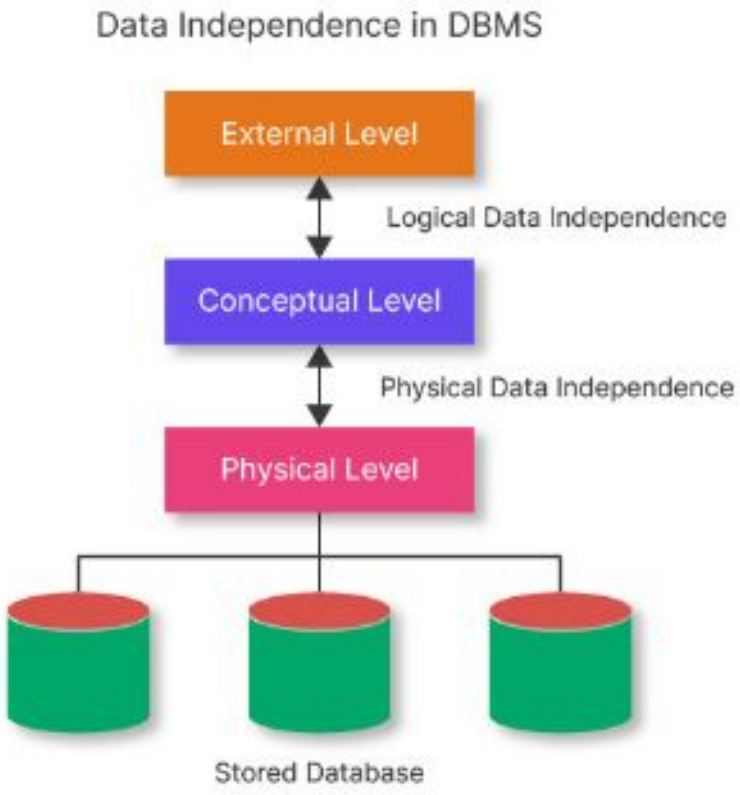
- **System analysts and application developers**
  - **System analysts:**
  - They understand the user requirements of naïve and sophisticated users and design applications including canned transactions to meet those requirements.
  - **Example:**
  - A system analyst gathers requirements for an inventory management system and ensures the database aligns with the operational needs.

  - **Application programmers**
  - Implement the specifications developed by analysts.
  - Developers write programs or applications that interact with the database. They use programming languages and query the database using SQL
  - **Example:**
  - A software developer writes a Python script that retrieves product details from the database and displays them on an e-commerce website.

Database users: Actors behind the scene

- **System designers and implementors:**
  - Design and implement DBMS packages in the form of modules and interfaces and test and debug them.
  - The DBMS must interface with applications, language compilers, operating system components, etc.
- **Tool developers**:
  - Design and implement software systems called tools for modeling and designing databases, performance monitoring, prototyping, test data generation, user interface creation, simulation etc. that facilitate building of applications and allow using database effectively.
- **Operators and maintenance personnel:**
  - They manage the actual running and maintenance of the database system hardware and software environment.

Data Independence



Data independence

Definition:

- The ability of the data to change the schema at one level of the database without having to change the schema at the next higher level

<u>In simple words:</u>

- It allows the User or Database Administrator to change the schema at one level without affecting the data or schema at another level.

- There are two levels of data independence in DBMS:
  - **Physical level data independence**
  - **Logical level data independence**

1. Physical Level Data Independence

- Physical Data Independence can be defined as the ability to change the physical level without affecting the logical or Conceptual level.
- Physical data independence gives us the freedom to modify the - Storage device, File structure, location of the database, etc. without changing the definition of conceptual or view level.
- Example: For example, if we take the database of the banking system and we want to scale up the database by changing the storage size and also want to change the file structure, we can do it without affecting any functionality of logical schema.
- Below changes can be done at the physical layer without affecting the conceptual layer -
  - Changing the storage devices like SSD, hard disk and magnetic tapes, etc.
  - Changing the access technique and modifying indexes.
  - Changing the compression techniques or hashing algorithms.

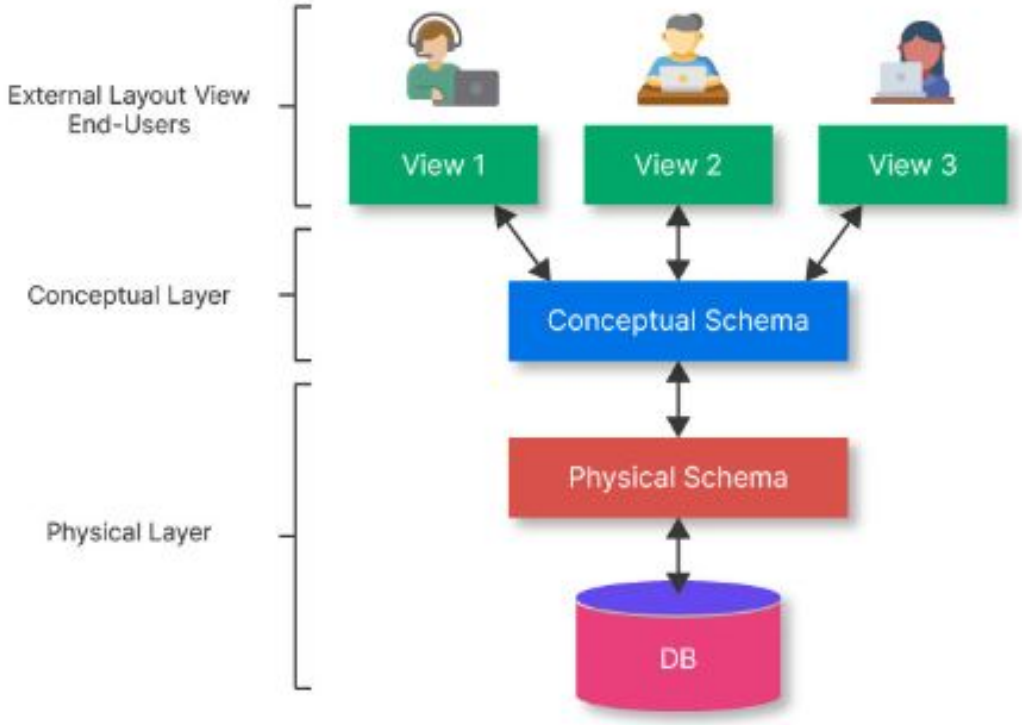
2. Logical Level Data Independence

- Logical Data Independence is a property of a database that can be used to change the logic behind the logical level without affecting the other layers of the database.
- Logical data independence is usually required for changing the conceptual schema without having to change the external schema or application programs.
- It allows us to make changes in a conceptual structure like adding, modifying, or deleting an attribute in the database.

- Example:
- If there is a database of a banking system and we want to add the details of a new customer or we want to update or delete the data of a customer at the logical level data will be changed but it will not affect the Physical level or structure of the database.
- Adding a New Attribute:
- Suppose a database table Employee initially has the attributes EmployeeID, Name, and Department. If the attribute Salary is added to the logical schema, applications querying EmployeeID and Name will still work without modification.

Data abstraction

- **Achieving Data Independence in DBMS Through Data Abstraction**
- Data Abstraction can be defined as extracting the necessary data by ignoring the remaining irrelevant details.

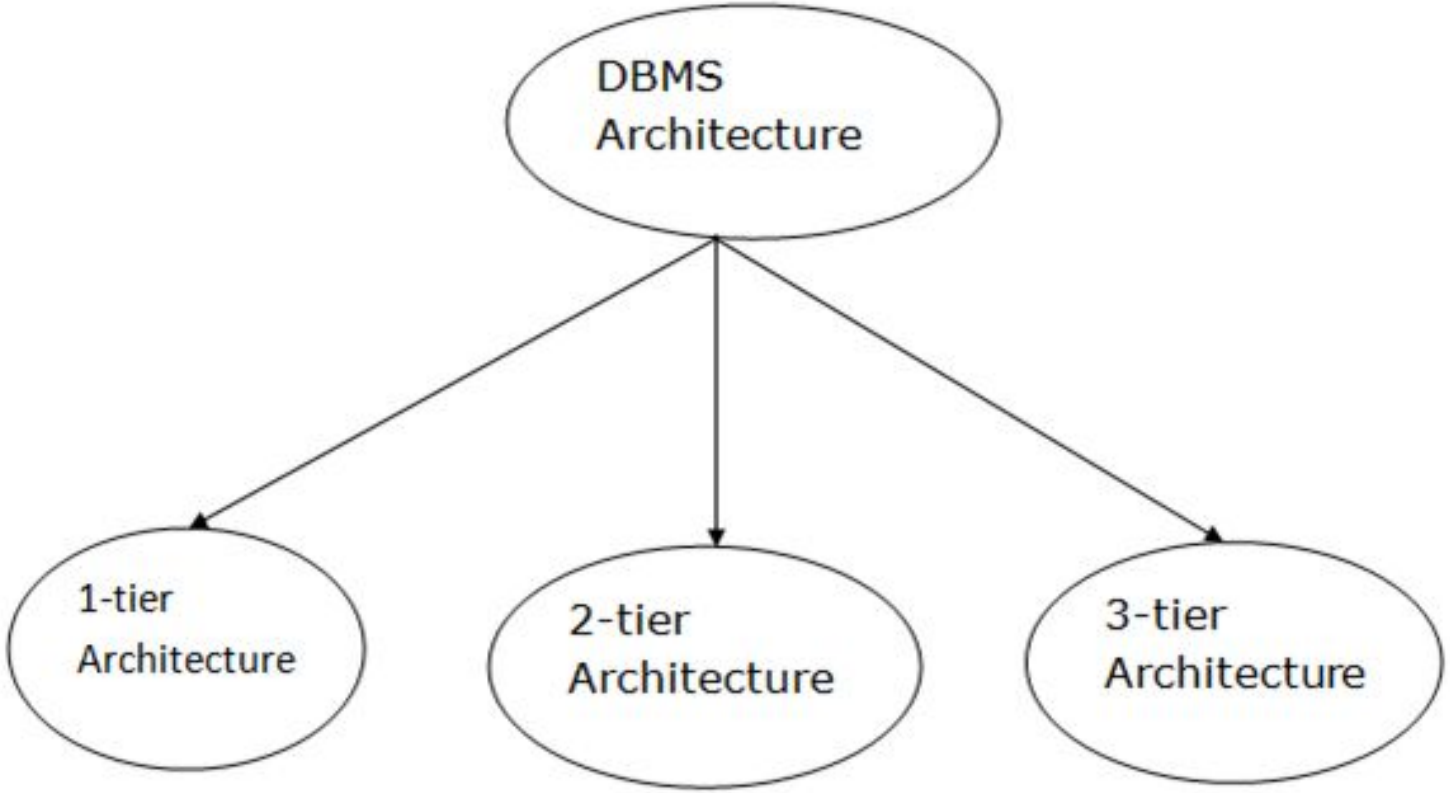


- Example: ATM

There are three levels of abstraction

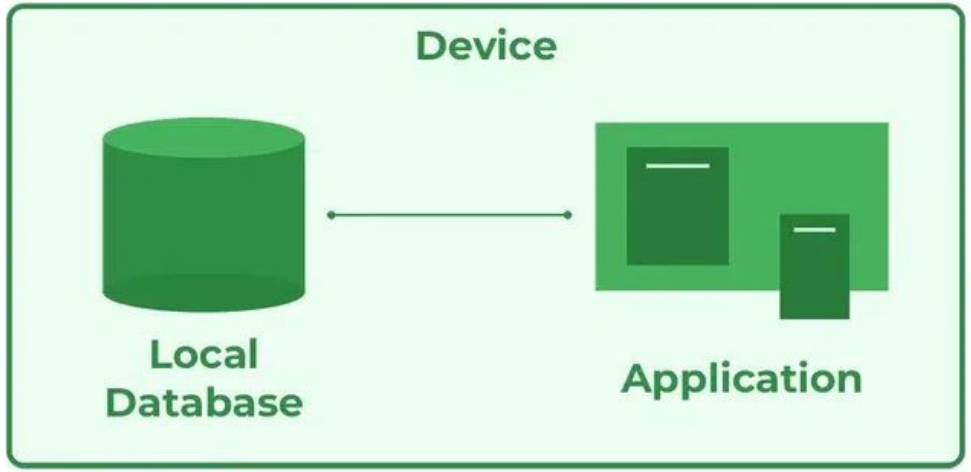
- **Physical or Internal Level -**
- Physical level is the lowest level of data abstraction, and It indicates how the data will be stored and describes the complex data structures and access methods to be used by the database.
- The internal level is used to describe the entire database architecture.
- **Conceptual or Logical Level -**
- The separation of the conceptual view from the internal view enables us to provide a logical description of the database concepts without the need to specify physical structures.
- The conceptual level comes between the physical level and the view level.
- It provides the link between the external schema and the internal schema of the database.
- **External or View Level -** It is the highest level of data abstraction.
- The external level describes the user interaction with the centralized database management system.
- This level is used to provide a Graphical User Interface to the user, and the user does not know about the file structure, access method, and other internal details of the database.

Types of DBMS Architecture



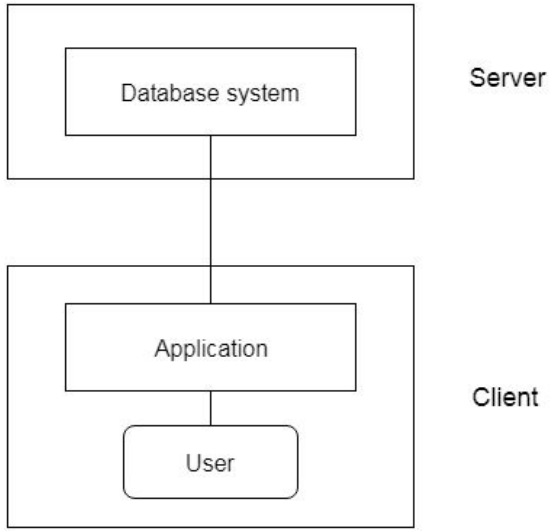
1-Tier Architecture

- In this architecture, the database is directly available to the user. It means the user can directly sit on the DBMS and uses it.
- Any changes done here will directly be done on the database itself. It doesn't provide a handy tool for end users.
- The 1-Tier architecture is used for development of the local application, where programmers can directly communicate with the database for the quick response.



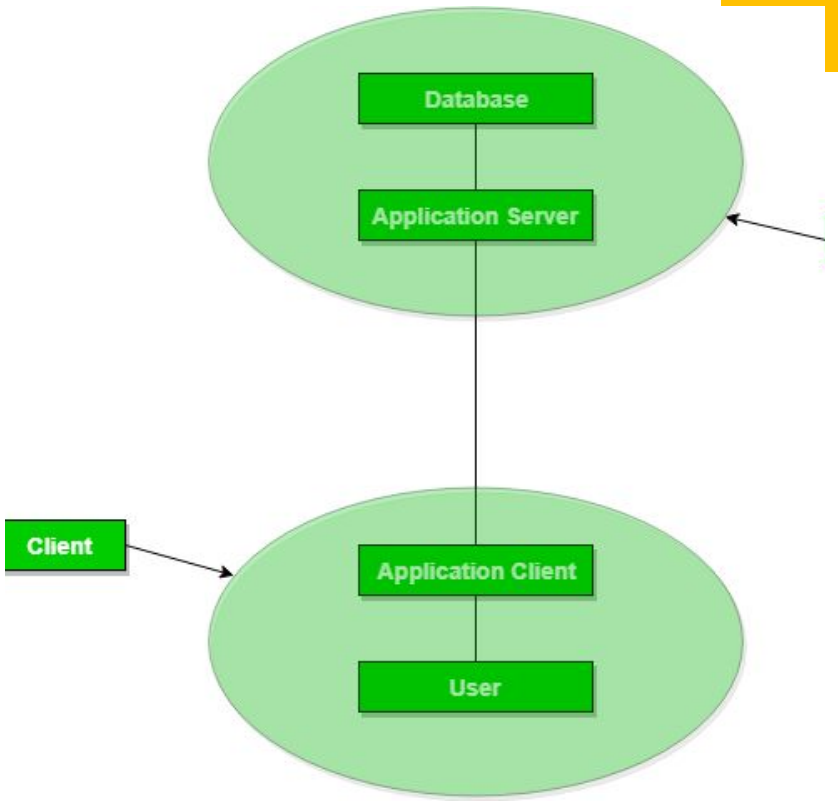
2-Tier Architecture

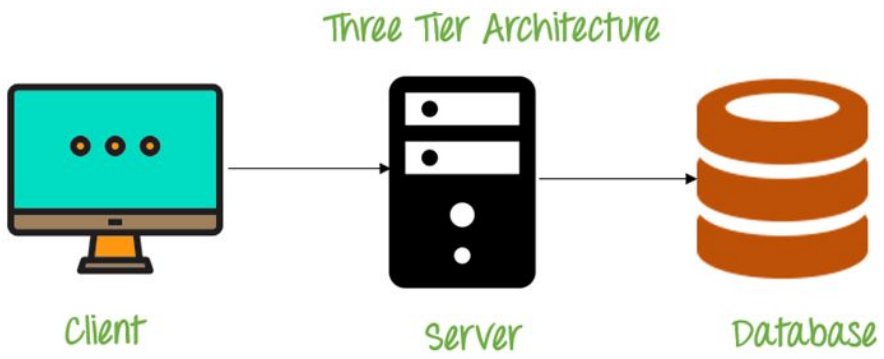
- The 2-tier architecture is similar to a basic client-server model.
- In the two-tier architecture, applications on the client end can directly communicate with the database at the server side. For this interaction, API's like: **ODBC**, **JDBC** are used.
- The user interfaces and application programs are run on the client-side.
- The server side is responsible to provide the functionalities like: query processing and transaction management.
- To communicate with the DBMS, client-side application establishes a connection with the server side.



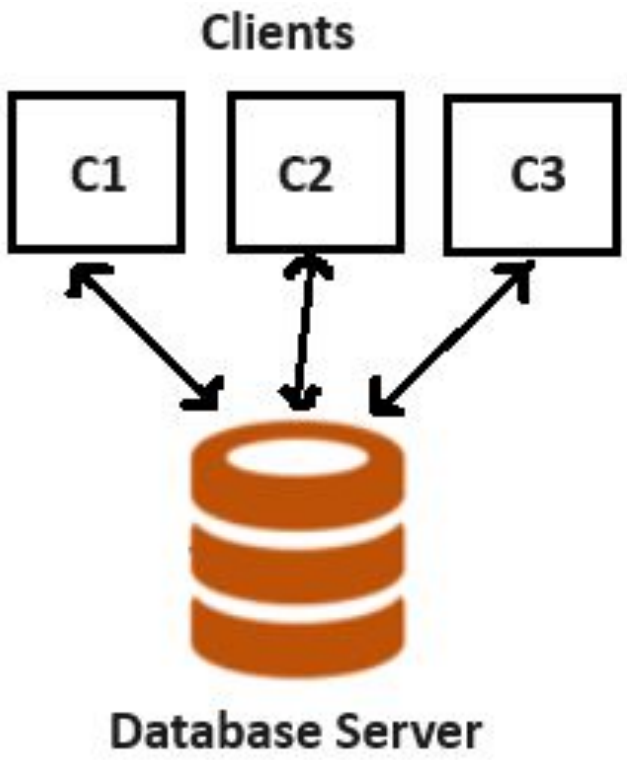
3-Tier Architecture

- The 3-Tier architecture contains another layer between the client and server.
- In this architecture, client can't directly communicate with the server.
- The application on the client-end interacts with an application server which further communicates with the database system.
- End user has no idea about the existence of the database beyond the application server. The database also has no idea about any other user beyond the application.
- The 3-Tier architecture is used in case of large web application.

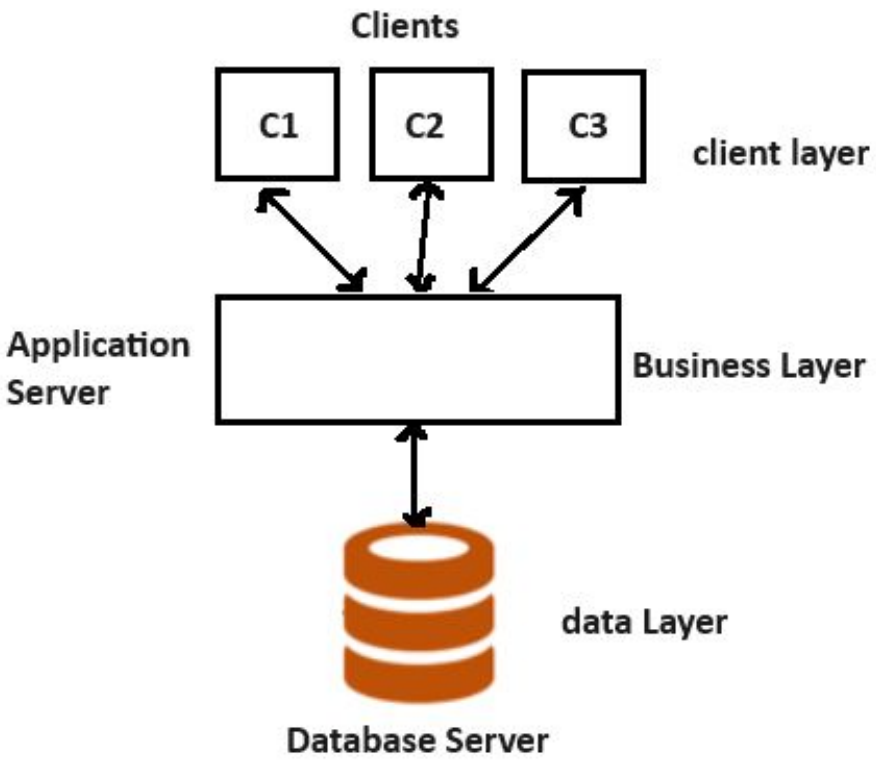




2-Tier


Clients
C1
C2
C3
Database Server

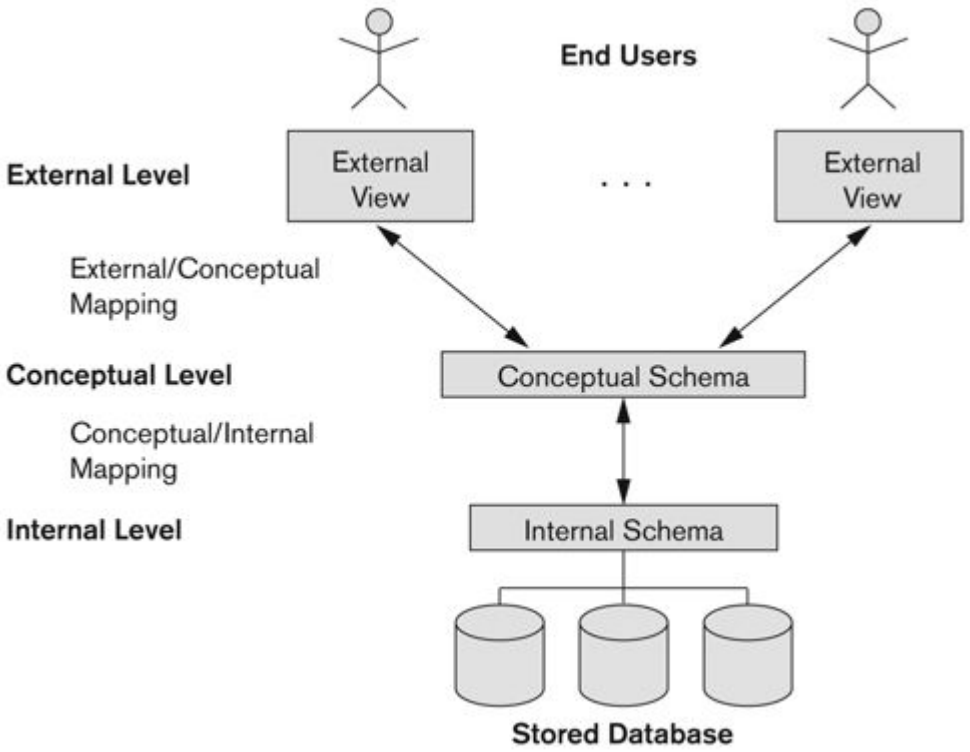
3-Tier


Clients
C1
C2
C3
client layer
Application Server
Business Layer
data Layer
Database Server

Three schema Architecture

Defines DBMS schemas at ***three*** levels:

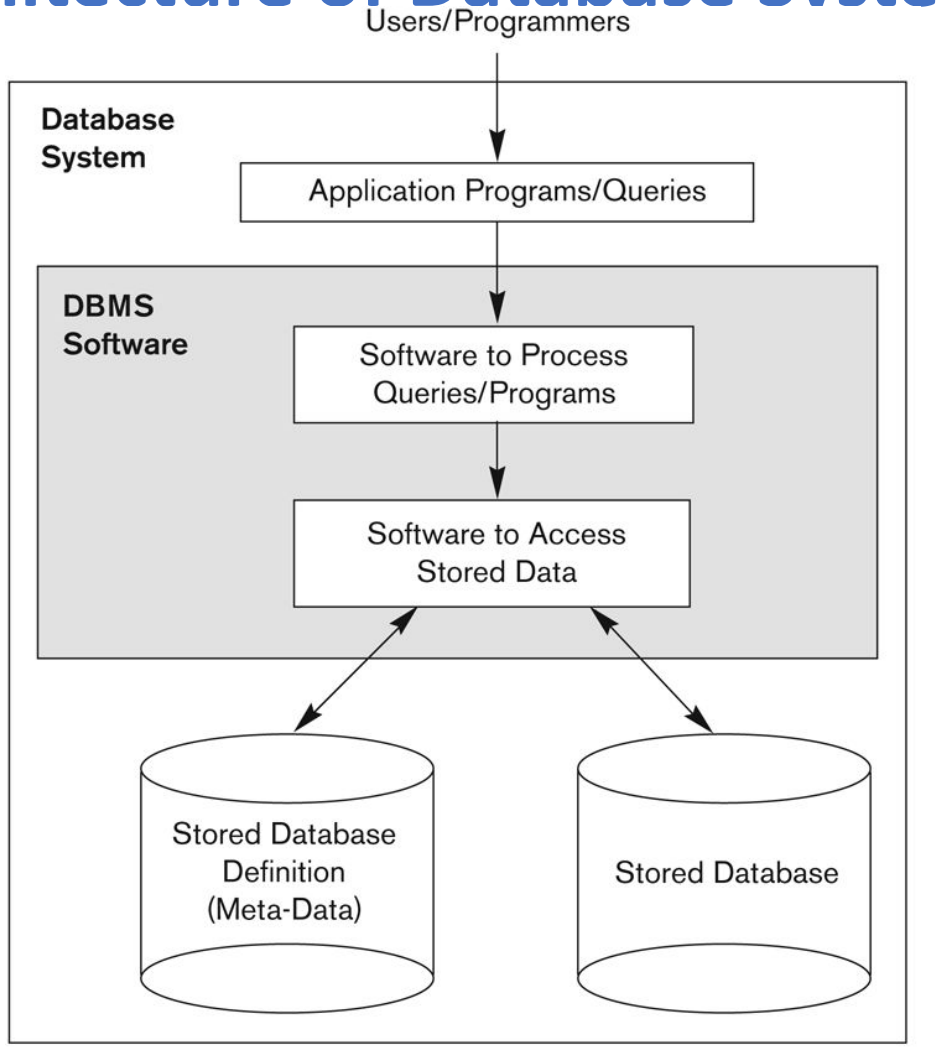
- **Internal schema** at the internal level to describe physical storage structures and access paths (e.g. indexes).
  - Typically uses a **physical** data model.
- **Conceptual schema** at the conceptual level to describe the structure and constraints for the whole database for a community of users.
  - Uses a **conceptual** or an **implementation** data model.
- **External schemas** at the external level to describe the various user views.
  - Usually uses the same data model as the conceptual schema.



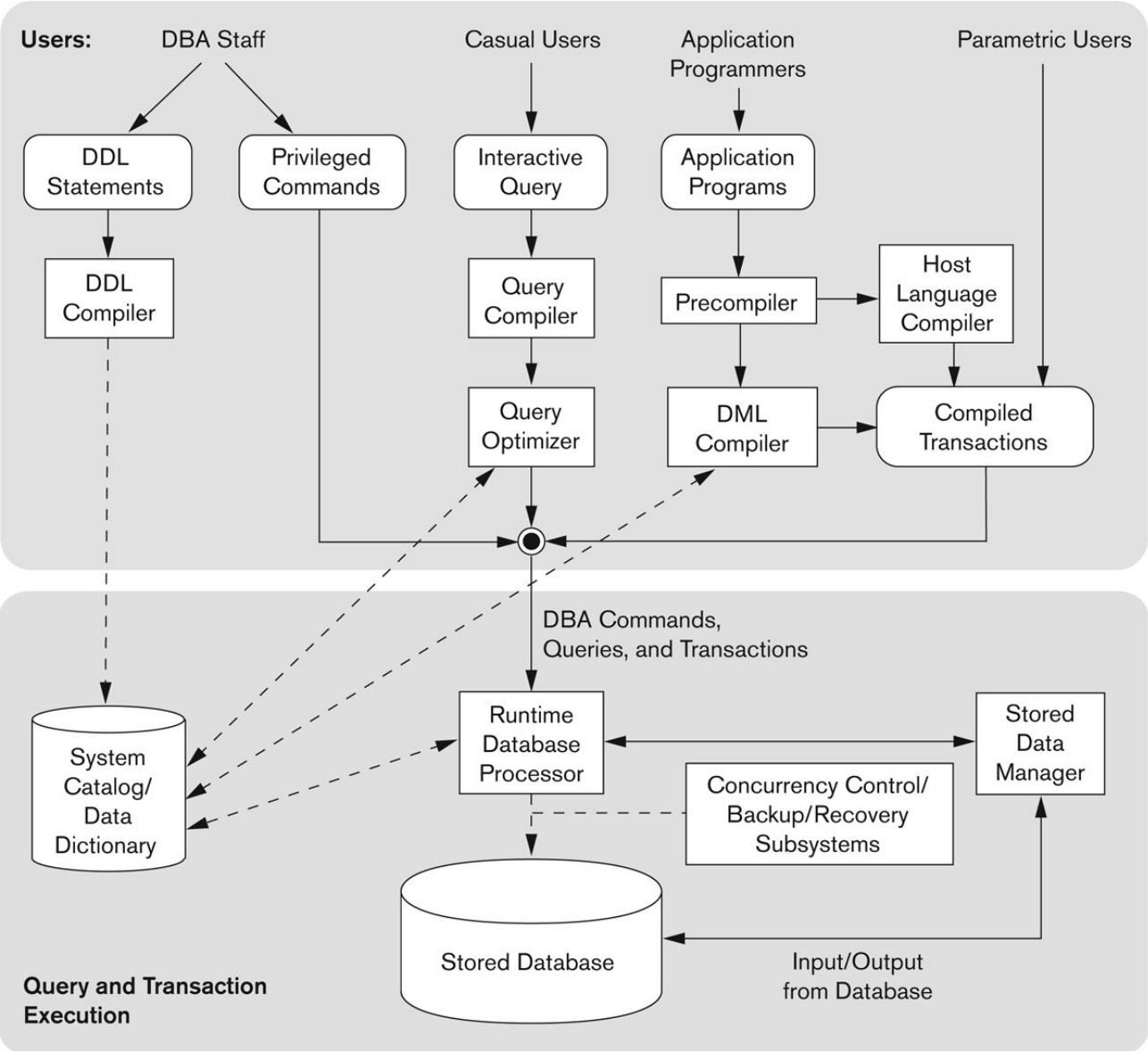
Three schema Architecture mapping

- Mapping is used to transform the request and response between various database levels of architecture.
- Mapping is not good for small DBMS because it takes more time.
- In External / Conceptual mapping, it is necessary to transform the request from external level to conceptual schema.
- In Conceptual / Internal mapping, DBMS transform the request from the conceptual to internal level.

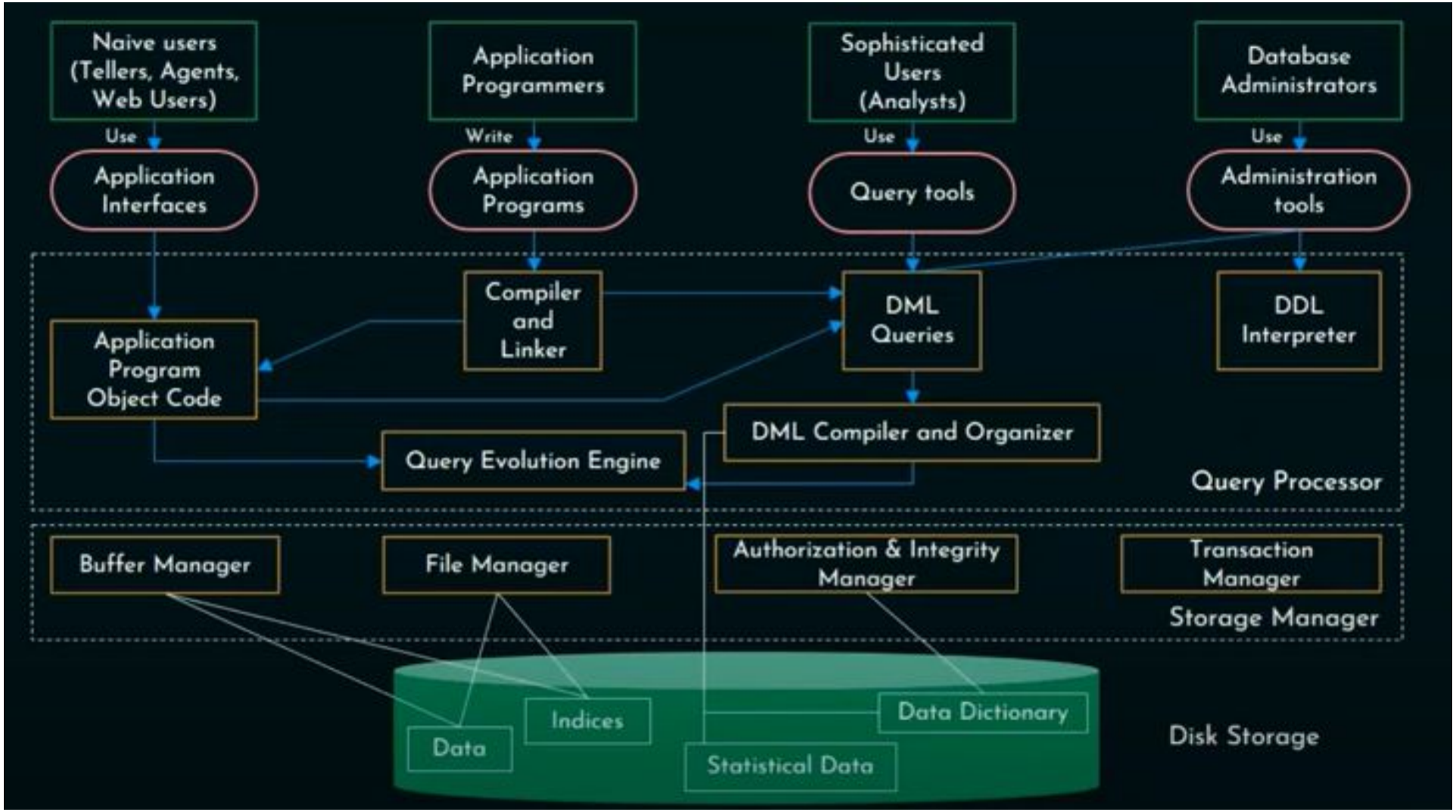
A simplified architecture of Database System



A simplified architecture of Database System



A simplified architecture of Database System



The Storage Manager

- ☆ Low level data stored - Storage Manager - Application Programs.
- ☆ Queries submitted to the system.
- ☆ Interaction with file manager.
- ☆ Raw data are stored on the disk using file system provided by OS.
- ☆ Translates various DML statements into low-level file system commands.

The Storage Manager

Components:

- ☆ Authorization and integrity manager.
- ☆ Transactions manager.
- ☆ File manager.
- ☆ Buffer manager.

Data structures:

- ☆ Data files.
- ☆ Data dictionary.

- Buffer manager-cache the data on priority. Handle memory management
- File manager- disk space management and data structure allocation
- Authorization and integrity manager
- Transaction Manager

Storage Management

The Query Processor

- ☆ DDL Interpreter.
- ☆ DML Compiler.
- ☆ Query Evaluation Engine.

Data Model

- **Data Model:**
  - A set of concepts to describe the ***structure*** of a database, the ***operations*** for manipulating these structures, and certain ***constraints*** that the database should obey.
- **Data Model Structure and Constraints:**
  - Constructs are used to define the database structure
  - Constructs typically include ***elements*** (and their ***data types***) as well as groups of elements (e.g. ***entity, record, table***), and ***relationships*** among such groups
  - Constraints specify some restrictions on valid data; these constraints must be enforced at all times
- **Data Model Operations:**
  - These operations are used for specifying database retrievals and updates by referring to the constructs of the data model.

Operations on the data model may include **basic model** operations (e.g. generic insert, delete, update) and **user-defined** operations (e.g. compute_student_gpa, update_inventory)

Data Model Categories

Conceptual (high-level, semantic) data models:

Provide concepts that are close to the way many users perceive data.

(Also called ***entity-based*** or ***object-based*** data models.)

Physical (low-level, internal) data models:

Provide concepts that describe details of how data is stored in the computer. These are usually specified in an ad-hoc manner through DBMS design and administration manuals

Implementation (representational) data models:

Provide concepts that fall between the above two, used by many commercial DBMS implementations (e.g. relational data models used in many commercial systems).